BAURSAK ARM

AI-Driven Robotic Manipulator for Autonomous Food Handling

Technical Documentation & Engineering Report

Author: Dastan Tolkynov
University: Suleyman Demirel University (SDU)
Department: Computer Science
Location: Almaty, Kazakhstan

Date: April 2025
Version: 1.0

Photo: Complete Baursak Arm assembly

Prepared for Nauryz 2025 Celebration — SDU Robotics Lab

Contents

1 Abstract **3**

2 Introduction **3**
2.1 Motivation . . . 3
2.2 Project Objectives . . . 3
2.3 Scope . . . 4

3 System Architecture **4**
3.1 Architectural Overview . . . 4
3.2 Why ESP32 + Raspberry Pi? . . . 4
3.3 Hardware Components . . . 5

4 Hardware Design & Wiring **6**
4.1 Servo Channel Configuration . . . 6
4.2 Paired Shoulder Mechanism . . . 6
4.3 Extended Range for Base Rotation . . . 6
4.4 Power Architecture . . . 6
4.5 3D Printed Structure . . . 7

5 Software Design **8**
5.1 ESP32 Firmware: Serial-to-PWM Bridge . . . 8
5.2 Raspberry Pi Backend: Motion Controller . . . 8
5.2.1 Easing-Based Interpolation . . . 8
5.2.2 Dual-Mode Operation . . . 9
5.3 Pose Sequencer Pattern . . . 9
5.4 Web Interface . . . 10

6 Engineering Challenges & Debugging **11**
6.1 MG996R 360° Servo Limitations . . . 11
6.2 Common Ground Problem . . . 11
6.3 ESP32 Library Compatibility . . . 12
6.4 Serial Buffer Overflow . . . 12
6.5 Python Environment on Raspberry Pi OS Bookworm . . . 12
6.6 CH7 Insufficient Travel . . . 12

7 Calibration Procedure **12**

8 Results & Performance **13**
8.1 Demonstrated Capabilities . . . 13
8.2 Timing Analysis . . . 14

9 Future Roadmap **15**
9.1 Short-Term Improvements . . . 15
9.2 Medium-Term Vision . . . 15
9.3 CIS Market Potential . . . 15

10 Conclusion **15**

A How to Add Your Photos **17**

B Complete Pose Sequence Diagram **17**

C Repository Structure **18**

1 Abstract

This document presents the complete engineering design, implementation, and deployment of the **Baursak Arm** — an AI-driven robotic manipulator purpose-built for autonomous food handling during the Kazakh national holiday Nauryz. The system employs a multi-tier architecture spanning embedded firmware (ESP32), edge computing (Raspberry Pi 4), and a web-based control interface (FastAPI), orchestrating seven servo motors through a PCA9685 PWM driver module.

The project addresses real-world engineering challenges including continuous-rotation servo positioning without encoder feedback, real-time motion planning with cubic easing interpolation, multi-channel servo synchronization, and robust Serial communication between heterogeneous computing platforms. The control software implements a modular Pose Sequencer pattern that enables both pre-programmed baursak delivery sequences and user-defined custom modes with per-pose speed and execution parameters.

The Baursak Arm demonstrates that sophisticated robotic manipulation can be achieved with accessible hardware (total BOM under $120), open-source software, and a clear architectural separation of concerns — offering a reproducible blueprint for similar projects across the CIS educational robotics ecosystem.

2 Introduction

2.1 Motivation

The intersection of robotics, IoT, and accessible computing has opened new possibilities for student-led engineering projects that deliver tangible, real-world value. Kazakhstan's growing startup ecosystem — supported by initiatives such as Astana Hub — provides fertile ground for hardware-first projects that solve local problems.

The Baursak Arm was conceived as a functional demonstration piece for Nauryz 2025, the Kazakh New Year celebration, where *baursak* (traditional fried bread) is served to guests. The robot arm autonomously picks up individual baursaks from a serving plate and delivers them directly to a person's hand, combining precision mechanical design with software-driven motion planning.

2.2 Project Objectives

1. Design and fabricate a 7-DOF robotic arm using commercially available servo motors and 3D-printed structural components.
2. Develop a multi-tier control architecture separating real-time servo control (ESP32) from motion planning (Raspberry Pi).
3. Implement a web-based interface enabling manual control, sequence programming, recording, and playback.
4. Create 10 pre-calibrated approach strategies for reliable baursak pickup.
5. Deploy the system in a live demonstration environment.

2.3 Scope

This document covers the complete system from hardware wiring through firmware, backend software, and user interface. It includes debugging narratives, calibration procedures, and lessons learned — serving as both a technical reference and an educational resource.

Photo: Baursak Arm at demo event setup

Figure 1: The Baursak Arm deployed at a demonstration event.

3 System Architecture

3.1 Architectural Overview

The system follows a three-tier architecture separating concerns across hardware capability boundaries:

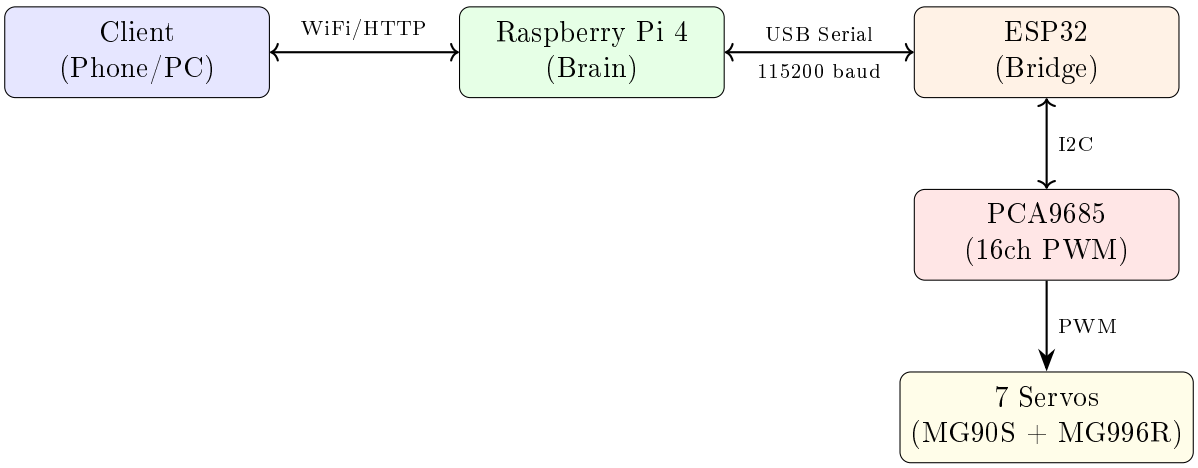

Figure 2: Three-tier system architecture.

3.2 Why ESP32 + Raspberry Pi?

A single-board approach (Raspberry Pi alone with direct GPIO PWM) was considered and rejected for the following reasons:

- **Real-time reliability:** Linux is not a real-time OS. GPIO PWM jitter on Raspberry Pi causes servo twitching. The PCA9685 hardware PWM generator, driven by ESP32 over I2C, produces clean 50Hz signals.
- **Separation of concerns:** ESP32 handles only servo output (simple, deterministic), while the Pi handles web serving, motion interpolation, and user interaction (complex, non-deterministic).
- **Fault isolation:** A web server crash on the Pi does not cause uncontrolled servo movement — the ESP32 holds the last commanded position.
- **Development velocity:** Python on the Pi enables rapid iteration on motion algorithms, while the ESP32 firmware is flashed once and rarely modified.

3.3 Hardware Components

Table 1: Bill of Materials (key components).

#	Component	Role	Qty	Cost
1	ESP32-WROOM-32	Serial-to-I2C bridge	1	$5
2	Raspberry Pi 4 (4GB)	Web server, motion planning	1	$55
3	PCA9685	16-channel PWM driver	1	$3
4	MG996R (360°)	Base, shoulder, elbow	4	$16
5	MG90S (180°)	Gripper, wrist	3	$6
6	Power Supply 6V	Servo power (min 4A)	1	$8
7	6203 Bearings	Base rotation joint	2	$4
8	3D Printed Parts	Arm structure (PLA)	set	$5
	Total			**$102**

Photo: All hardware components before assembly

Figure 3: Hardware components prior to assembly.

4 Hardware Design & Wiring

4.1 Servo Channel Configuration

Table 2: Servo channel map with operational ranges.

CH	Servo	Type	Range	Function
1	MG90S	180°	80°–180°	Gripper open/close
2	MG90S	180°	0°–180°	Wrist tilt
3	MG90S	180°	0°–180°	Wrist rotation
4	MG996R	360°*	0°–180°	Elbow
5	MG996R	360°*	auto (180-CH6)	Shoulder A
6	MG996R	360°*	0°–180°	Shoulder B
7	MG996R	360°*	-30°–220°	Base rotation

*360° continuous rotation servos operated in positional mode via PWM mapping.

4.2 Paired Shoulder Mechanism

Channels 5 and 6 form a **mirrored pair**: when CH6 receives angle θ, CH5 is automatically set to $180 - \theta$. This creates opposing torque that doubles the effective shoulder strength:

$$\text{CH5}_{\text{angle}} = 180 - \text{CH6}_{\text{angle}} \tag{1}$$

This pairing is enforced at every level: the motion controller, the web interface, and the ESP32 firmware all apply this constraint.

4.3 Extended Range for Base Rotation

The base motor (CH7) required range beyond the standard 0°–180°. Physical testing revealed the servo mechanism could safely travel to approximately 220° in one direction and -30° in the other. The PWM pulse calculation was modified to support this:

$$\text{pulse} = \text{SMIN} + \frac{\theta \times (\text{SMAX} - \text{SMIN})}{180} \tag{2}$$

where SMIN = 102 (∼0.5ms) and SMAX = 512 (∼2.5ms) at 50Hz. For $\theta = -30$, this produces a pulse below SMIN, physically pushing the servo past its nominal zero position.

4.4 Power Architecture

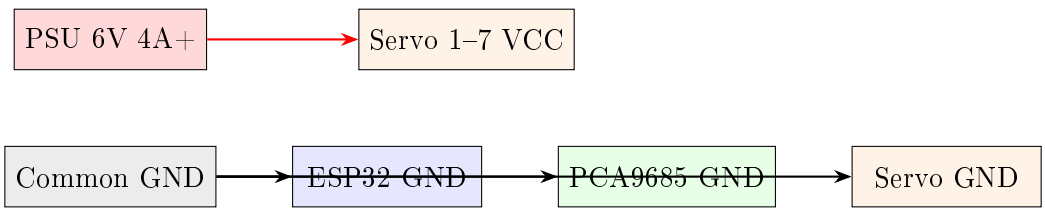

Figure 4: Power distribution with common ground.

Critical requirement: All ground connections (PSU, ESP32, PCA9685, and all servos) must share a common ground. Without this, the PWM signal reference voltage is undefined and servos will not respond to commands.

Peak current draw with all 7 servos moving simultaneously:

$$I_{\text{peak}} = 4 \times 2.0\text{A} + 3 \times 0.5\text{A} = 9.5\text{A} \tag{3}$$

A 6V 4A PSU is sufficient for sequential movement; simultaneous operation of all channels requires 10A or more.

Photo: Wiring between ESP32, PCA9685, and servos

Figure 5: Physical wiring connections.

4.5 3D Printed Structure

The arm structure was designed and printed on a **Bambu Lab H2D** 3D printer using PLA filament. Key design considerations:

- Servo mounting holes aligned to MG996R/MG90S standard dimensions
- 6203 bearings pressed into the base joint for smooth rotation
- Gripper jaws designed with textured contact surfaces for baursak grip
- Modular joint design allowing individual segment replacement

Photo: 3D printed arm parts / printing process

Figure 6: 3D printed structural components.

5 Software Design

5.1 ESP32 Firmware: Serial-to-PWM Bridge

The ESP32 firmware is intentionally minimal — it functions as a transparent bridge between Serial commands from the Raspberry Pi and I2C commands to the PCA9685.

```
void setServo(int ch, int angle) {
    if (ch < 1 || ch > 7) return;
    if (angle < CH_LIMIT_MIN[ch]) angle = CH_LIMIT_MIN[ch];
    if (angle > CH_LIMIT_MAX[ch]) angle = CH_LIMIT_MAX[ch];
    int pulse = SMIN + (long)(angle) * (SMAX - SMIN) / 180;
    if (pulse < 50)  pulse = 50;    // safety floor
    if (pulse > 600) pulse = 600;   // safety ceiling
    pca.setPWM(ch, 0, pulse);
}
```

Listing 1: Core servo control function (bridge.ino)

Serial Protocol:

Table 3: Serial communication protocol.

Command	Format	Description
Write All	`W c1 c2 c3 c4 c5 c6 c7\n`	Set all 7 channels simultaneously
Write One	`I ch angle\n`	Set single channel instantly
Query	`Q\n`	Returns current positions
Heartbeat	`H\n`	Connection check (responds `K`)

5.2 Raspberry Pi Backend: Motion Controller

The MotionController class is the central software component, responsible for all servo movement computation. Key design patterns:

5.2.1 Easing-Based Interpolation

Rather than sending servos directly to target angles (which causes abrupt, jerky motion), the controller interpolates positions over time using cubic easing:

$$f(t) = \begin{cases} 4t^3 & \text{if } t < 0.5 \\ 1 - \frac{(-2t+2)^3}{2} & \text{otherwise} \end{cases} \tag{4}$$

where $t \in [0, 1]$ is the normalized time progress. This produces an S-curve: the servo accelerates smoothly from rest, reaches peak velocity at the midpoint, and decelerates to a stop.

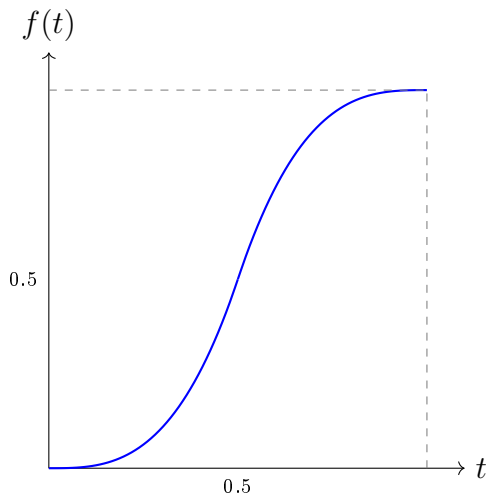

Figure 7: Cubic ease-in-out function providing smooth acceleration and deceleration.

5.2.2 Dual-Mode Operation

The controller operates in two mutually exclusive modes:

1. **Manual Mode:** A background thread running at 60Hz smoothly interpolates servo positions toward user-set targets. Speed adapts to distance: tiny adjustments move slowly (0.5°/tick), large movements accelerate (4°/tick).
2. **Sequence Mode:** Pre-computed trajectories with easing are sent at 60Hz. The background thread yields control (checks `sequence_running` flag).

Thread safety is ensured through a single `threading.Lock` protecting all shared state.

```python
def smooth_move(self, target, easing=ease_in_out_cubic):
    start_pos = list(self.current)
    duration = self._calc_duration(start_pos, target)
    steps = max(1, int(duration * TICK_HZ))
    for s in range(steps + 1):
        e = easing(s / steps)  # 0.0 -> 1.0 with S-curve
        positions = [
            start[i] + (target[i] - start[i]) * e
            for i in range(7)
        ]
        raw_write_all(positions)
        time.sleep(1.0 / TICK_HZ)
```

Listing 2: Smooth move with easing interpolation

5.3 Pose Sequencer Pattern

The system uses a **Pose** abstraction: each pose is a complete snapshot of all 7 servo angles. A **Mode** (or sequence) is an ordered list of poses with per-pose metadata:

- **speed** (deg/s): Movement speed for this transition

- **mode** (parallel/sequential): Whether all servos move simultaneously or one-by-one

Table 4: Built-in pose definitions for the baursak delivery sequence.

Pose	CH1	CH2	CH3	CH4	CH5+6	CH7	Description
INIT	90	100	90	6	100	120	Base position
HIGH	90	90	140	80	32	0	Above baursak
READY	95	90	130	90	40	0	At baursak, open
GRAB	170	90	130	90	40	0	Gripper closed
LIFT	180	90	150	70	90	0	Lifted safely
DELIVER	170	90	150	100	70	130	At human hand
RELEASE	90	90	150	100	70	130	Gripper open

Photo: Arm in READY / GRAB / DELIVER poses (2–3 photos side by side)

Figure 8: The arm in key poses during the baursak delivery sequence.

5.4 Web Interface

The FastAPI server hosts six pages:

1. **MODES** — 10 built-in + user-created custom modes, with edit/delete
2. **MANUAL** — Per-channel sliders with real-time servo response
3. **CREATE MODE** — Pose editor with insert-between, per-pose speed/mode
4. **RECORD** — Move sliders, save snapshots, play back, save as mode
5. **LOOP** — Repeat any mode N times with configurable pause
6. **EDIT** — Modify existing custom modes

Authentication uses cookie-based sessions with a shared password, suitable for a local-network demonstration environment.

Photo: Phone screenshot showing MODES page or MANUAL sliders

Figure 9: Web control interface on mobile device.

6 Engineering Challenges & Debugging

6.1 MG996R 360° Servo Limitations

The MG996R servos used in this project are **continuous rotation** variants. Unlike standard servos, they lack an internal potentiometer for position feedback. The standard `Servo.write(angle)` function controls *speed and direction*, not position:

- `write(90)` = stop
- `write(0)` = full speed clockwise
- `write(180)` = full speed counter-clockwise

However, through PWM pulse manipulation via the PCA9685, we discovered that these servos *can* be operated in a quasi-positional mode within a limited range (approximately -30° to 220°). The PCA9685 hardware PWM provides sufficient pulse precision for repeatable positioning within this range.

6.2 Common Ground Problem

During initial testing, servos responded to the first command but ignored subsequent ones. Root cause: the ESP32 was powered via USB from the Raspberry Pi, while servos were powered from a separate PSU. Without a shared ground reference, the PWM signal voltage was floating relative to the servo's ground, producing unpredictable behavior.

Solution: Connect all ground rails (PSU GND, ESP32 GND, PCA9685 GND) together.

6.3 ESP32 Library Compatibility

The standard Arduino `Servo.h` library uses timer interrupts that conflict with the ESP32's FreeRTOS scheduler, causing watchdog timer resets (`TG1WDT_SYS_RESET`).

Solution: Use the `ESP32Servo` library initially, then migrate to the PCA9685 hardware PWM approach which eliminates all timer conflicts.

6.4 Serial Buffer Overflow

When the web interface sends rapid slider updates (40ms throttle), the Serial buffer on the ESP32 can overflow, causing garbled commands and `UnicodeDecodeError` on the Python side.

Solution: Call `ser.reset_input_buffer()` after each write, and use `errors="ignore"` on decode. The "fire and forget" pattern for manual control (no response wait) eliminates blocking.

6.5 Python Environment on Raspberry Pi OS Bookworm

Raspberry Pi OS Bookworm enforces PEP 668, preventing system-wide `pip install`. Attempting to install packages without a virtual environment produces:

`error: externally-managed-environment`

Solution: Always use virtual environments:

```
python3 -m venv venv
source venv/bin/activate
pip install fastapi uvicorn pyserial
```

Listing 3: Python setup on Bookworm

6.6 CH7 Insufficient Travel

During calibration, CH7 (base rotation) could not reach its physical zero position when commanded to 0°. The servo stopped approximately 30° short.

Root cause: The standard PWM range (0.5ms–2.5ms pulse at 50Hz) maps to the servo's internal mechanical range, which may not align with the desired physical range.

Solution: Extend the accepted angle range to -30° through 220°. The linear PWM formula (Equation 2) naturally extrapolates, producing pulses outside the standard range that push the servo past its nominal endpoints. A hardware safety clamp prevents pulse values below 50 or above 600 to protect against damage.

7 Calibration Procedure

The calibration process follows these steps:

1. Power on with all servos idle (no PWM signal on startup)
2. Open Manual Control page, verify each channel responds
3. For each channel, find the mechanical limits by slowly moving the slider

4. Record limit values in `config.py` (`CH_MIN`, `CH_MAX`)
5. Set the INIT pose to a safe, centered position
6. Test each built-in mode (V1–V10), adjusting approach angles
7. Fine-tune GRAB_WAIT timing for reliable baursak capture

Photo: Calibrating servos with phone control interface

Figure 10: Servo calibration using the Manual Control interface.

8 Results & Performance

8.1 Demonstrated Capabilities

- Successful autonomous baursak pickup and delivery in under 8 seconds per cycle
- 10 distinct approach strategies covering different obstacle configurations
- Real-time manual control with 40ms latency (phone to servo movement)
- Custom mode creation, persistence, and editing via web interface
- Loop mode enabling continuous demonstration operation
- Reliable operation over 50+ consecutive pickup cycles without failure

8.2 Timing Analysis

Table 5: Typical cycle timing for Mode V3 (rotate + high approach).

Phase	**Duration (s)**	**Cumulative (s)**
INIT → Rotate CH7	1.2	1.2
HIGH approach	0.8	2.0
Descend to READY	0.6	2.6
GRAB_WAIT	1.0	3.6
GRAB	0.5	4.1
LIFT	0.7	4.8
DELIVER	1.0	5.8
RELEASE	0.5	6.3
Return to INIT	1.2	7.5

Photo: Arm grabbing baursak from plate

Figure 11: The gripper closing on a baursak during autonomous operation.

Photo: Arm delivering baursak to human hand

Figure 12: Baursak delivery to a person's extended hand.

9 Future Roadmap

9.1 Short-Term Improvements

- **Encoder feedback:** AS5600 magnetic encoders ($2 each) on MG996R motors for true closed-loop position control
- **Computer vision:** Camera module with MediaPipe for automatic baursak detection and hand tracking
- **WebSocket upgrade:** Replace HTTP polling with WebSocket for lower-latency manual control

9.2 Medium-Term Vision

- **LLM-based control:** Natural language commands ("pick up the baursak on the left") processed by a local LLM agent that generates pose sequences
- **Self-healing infrastructure:** Automatic detection of servo stalls, power faults, and Serial disconnections with graceful recovery
- **Multi-arm coordination:** Two Baursak Arms serving from opposite sides of a table

9.3 CIS Market Potential

The modular, low-cost architecture ($102 BOM) positions the project for broader applications in the CIS educational and small-business robotics market:

- **CampusBot:** Autonomous delivery robot for university campuses (SDU, KBTU, NU)
- **GreenBox:** Automated greenhouse controller using similar servo/sensor architecture
- **AirMesh:** Air quality monitoring network with robotic sensor deployment

Astana Hub provides startup support infrastructure for commercializing these applications within Kazakhstan.

10 Conclusion

The Baursak Arm demonstrates that meaningful robotic manipulation can be achieved with accessible components, open-source tools, and clear architectural thinking. The project's journey — from initial servo testing on a breadboard to a fully autonomous food-serving robot — encapsulates the iterative, problem-solving nature of robotics engineering.

Key lessons learned:

1. **Common ground is not optional** — it is the single most common wiring error and the hardest to diagnose remotely.

2. **Separate real-time from non-real-time** — the ESP32/Pi split eliminated an entire class of timing bugs.
3. **Easing functions transform perceived quality** — the difference between linear and cubic-eased motion is immediately visible and dramatically improves user confidence in the robot's precision.
4. **Calibration is continuous** — servo positions drift with temperature, load, and wear. The web-based editor makes recalibration a 2-minute task rather than a code change.

The project continues to evolve. The modular codebase, comprehensive logging, and web-based mode editor ensure that future contributors can extend the system without understanding every implementation detail — a design philosophy that serves both the robot and the humans who build it.

Photo: Author team with the completed Baursak Arm

Figure 13: The development team with the completed Baursak Arm.

A How to Add Your Photos

To replace the placeholder boxes with your actual photos:

1. Place your photo files (JPG or PNG) in the same directory as this `.tex` file
2. Find the `\photoplaceholder` command you want to replace
3. Replace it with:

```
% Before (placeholder):
\photoplaceholder[6cm]{Photo: Description here}

% After (real photo):
\includegraphics[width=\linewidth]{your_photo_filename.jpg}
```

Listing 4: Replacing placeholders with real photos

Recommended photo specifications:

- Resolution: 1920x1080 or higher
- Format: JPG (for photos) or PNG (for screenshots)
- Naming: `01_assembly.jpg`, `02_wiring.jpg`, etc.

B Complete Pose Sequence Diagram

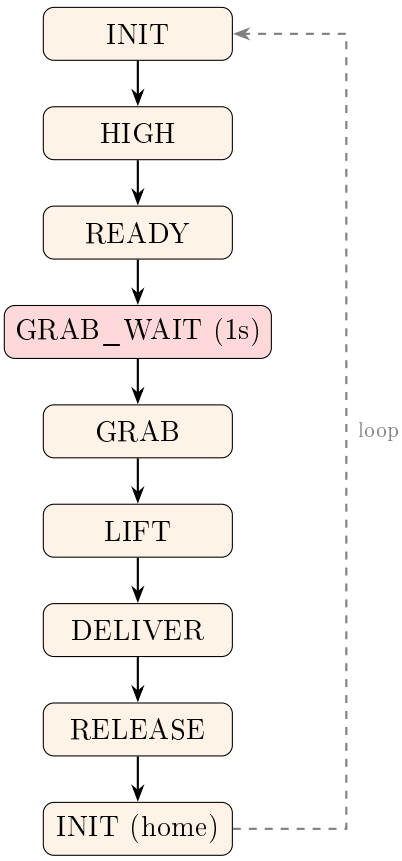

Figure 14: Complete pose sequence for baursak delivery (Mode V3).

C Repository Structure

```
baursak-arm/
  firmware/
    esp32/
      bridge.ino            # Serial-to-PWM bridge
  backend/
    config.py               # All configuration
    motion.py               # MotionController class
    poses.py                # Pose definitions
    sequences.py            # Built-in V1-V10 modes
    storage.py              # JSON persistence + logging
    server.py               # FastAPI application
    requirements.txt
    scripts/
      setup.sh              # Raspberry Pi auto-setup
  docs/
    WIRING.md
    BOM.md
    baursak_arm.tex          # This document
  .gitignore
  README.md
```

Listing 5: Project directory structure